Agent Workspace as Code

A Portable Governance Substrate for Production Agentic AI Workspaces

Growthub Local White Paper

May 18, 2026 foundation / May 21, 2026 workflow extension

@growthub/cli@0.13.1 / growthub-custom-workspace-starter-v1

Release-backed, npm-published, smoke-tested

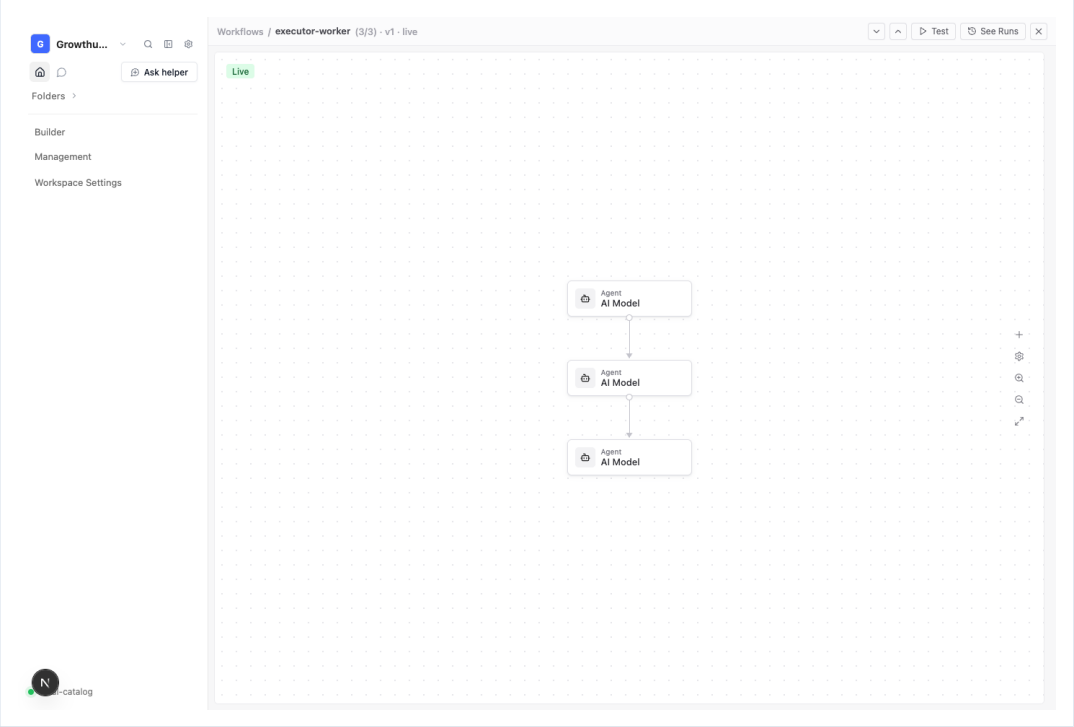

Figure 1. Workflow support extends the original AWaC substrate without replacing the core workspace artifact thesis.

Official optimized white paper snapshot: the original AWaC workspace mental model remains the center. The May 21 workflow release extends that model by making governed, sandbox-backed workflows first-class Builder items inside the same portable workspace artifact.

Introduction

This white paper is the publish-ready Growthub Local reference for Agent Workspace as Code. It preserves the original AWaC thesis while documenting the completed Builder, workflow, dashboard, helper, CLI, local model, sandbox, and governed fork surfaces that now ship through the open-source repository and npm-distributed starter kit.

The document is intentionally practical: it connects the user-facing product journey to the underlying release artifact, so readers can understand not only what the workspace does, but why the architecture is safe, portable, and governable.

Navigation

Project: https://www.growthub.ai/f/growthub-local

Author: LinkedIn / Antonio Romero | X / @ant0ni0_r0mer0

Table Of Summary

Core thesis	The workspace is the owned, portable, governed artifact for production agentic AI.
Frozen snapshot	A production workspace operating system fits inside a portable config artifact under 1 MB; the current validated workspace config is approximately 688 KB.
Distribution	Growthub CLI exports the governed starter and preserves fork, policy, trace, helper, dashboard, Data Model, workflow, and runtime state.
L1-L5 frame	Inputs are parsed into workspace state, routed through strategy, persisted atomically, and emitted as a human-and-agent-readable snapshot.
Workflow extension	Builder workflow items are user-facing folder objects over governed sandbox-environment execution rows.
Permissioned surface	Super admins decide what appears above the divider as user-facing navigation while governed operational depth remains available underneath.
Safety model	Draft changes save separately; Test runs the exact saved draft; Publish promotes only after successful execution.
No-code surface	Dashboards, widget sidecars, Data Model records, workflow nodes, Helper proposals, and local model configuration share the same workspace grammar.

Executive Summary

Agent Workspace as Code, or AWaC, is a practical operating model for production agentic AI: the workspace itself becomes the owned, portable, governed artifact.

Instead of treating agent work as a loose collection of prompts, scripts, dashboards, memory files, local folders, and hosted integrations, AWaC packages the working environment into a forkable application with configuration, user-facing navigation, governed Data Model objects, dashboards, views, traces, helper state, local runtime controls, and optional hosted authority.

The completed Growthub Local workspace starter proves this model in a concrete artifact. A live exported workspace carries dashboards, widget state, custom object views, folder navigation, helper traces, source records, sandbox/runtime objects, lifecycle metadata, and now workflow orchestration support inside the same governed workspace system.

The frozen snapshot is the proof: a complete workspace operating system remains portable as a config artifact under 1 MB, with the current validated workspace config at approximately 688 KB while still carrying dashboards, custom table views, folders, live widget state, helper memory, Data Model objects, controlled navigation, sandbox execution, and workflow orchestration.

The strategic breakthrough is not only technical portability. It is the user and admin mental model. Super admins decide what appears in the user workspace surface while keeping deeper operational primitives governed underneath. Users retain meaningful control through folders, dashboards, permitted object views, search, filters, customization, helper access, and workflow-specific shortcuts.

The permissioned surface model is the breakthrough. A customer sees a clean folder tree, dashboards, Management, and Workspace Settings. Under the hood, the same workspace still carries governed objects, rows, fields, dashboard canvas state, widgets, traces, helper threads, source records, local and hosted lifecycle state, and configuration rules.

AWaC complements frontier agent standards by defining the portable local artifact around them. It does not replace hosted agent platforms, MCP, tracing, guardrails, or observability. It gives them a durable user-owned workspace substrate.

L1-L5 Architecture: AWaC as the Recursive Workspace Snapshot

The Growthub Local implementation is a direct manifestation of the L1-L5 infrastructure-first pattern. The workspace is not only a UI and not only a repo. It is a recursive snapshot that accepts inputs, parses them into governed workspace structure, selects safe transformation strategies, persists atomic state, and emits a human-and-agent-readable product surface.

L1 INPUT (Token Validation): source repos, skills, worker kits, templates, prompts, Data Model schemas, sandbox rows, helper requests, dashboard definitions, workflow nodes, and CLI commands enter as validated workspace tokens.

L2 PARSING (Data Aggregation): the CLI and workspace builder aggregate those tokens into growthub.config.json, .growthub-fork identity, policy, trace, Data Model objects, dashboards, widget config, workflow orchestration config, helper threads, and runtime adapters.

L3 HEURISTICS (Strategy Selection): Growthub selects the correct path for the user journey: starter init, repo import, skill import, kit download, helper proposal, dashboard edit, workflow draft, sandbox test, publish, deploy check, fork heal, or hosted authority attach.

L4 EXECUTION (Atom Persistence): atomic state persists through the governed workspace artifact: PATCH /api/workspace for config, .growthub-fork/fork.json for identity, policy.json for operator constraints, trace.jsonl for lifecycle events, source records for helper/run evidence, and npm-distributed starter files for reproducible export.

L5 PRESENTATION (Snapshot Emission): the emitted snapshot is visible as the local Builder, folders rail, dashboards, Data Model, workflow canvas, sidecars, run traces, helper sidecar, CLI JSON envelopes, docs, and release tarball.

The recursive invariant is: S[k+1] = Freeze(Emit(Exec(Heur(Parse(Tokens[k+1] + D1/R1-R22 deltas))))). In Growthub Local, each release snapshot freezes the current workspace grammar while preserving open hooks for additive capabilities.

Growthub CLI, Export, and Governed Fork Lifecycle

The Growthub CLI is the distribution and lifecycle control plane for AWaC. The CLI turns sources into governed workspaces, exports the starter kit, validates skill and kit shape, registers fork identity, writes policy and trace, checks deployment readiness, and keeps the npm-distributed artifact reproducible.

The canonical user path starts with the CLI: npm create @growthub/growthub-local@latest, growthub starter init, growthub starter import-repo, growthub starter import-skill, or growthub kit download. Each path lands in the same artifact shape: growthub.config.json, apps/workspace, .growthub-fork, SKILL.md, AGENTS.md, helpers, skills, templates, workers, brands, and docs.

The governed fork is the durable identity layer. .growthub-fork/fork.json stores fork identity and base version. policy.json stores the operator contract. trace.jsonl records lifecycle events. project.md stores session memory. authority.json and agents/*.json attach hosted authority additively when needed.

This matters for open source because the repo does not merely document AWaC. The npm package exports it. The CLI tarball is the executable distribution path, and the starter kit is the materialized workspace that users can inspect, fork, run, deploy, and keep current.

Figure 2. Growthub CLI is the distribution and lifecycle control plane: discovery, starter export, governed fork setup, and agent-facing command paths.

1. The Problem: Agentic AI Needs a Workspace Substrate

Agentic AI systems differ from simple chatbots because they do more than generate text. They call tools, access external data, make multi-step decisions, maintain state, and can trigger real-world effects.

That creates operational demands: durable state, tool governance, user-visible and admin-hidden surfaces, traceability, controlled memory and knowledge persistence, human oversight, reproducible environments, deployable runtime surfaces, permissioned UI exposure, and local-to-hosted upgrade paths.

The gap is that most agent workspaces remain informal. Teams combine a repo, scripts, chat transcripts, cloud dashboards, local notes, and manually configured integrations. That can work for experiments. It does not scale into trustworthy agent operations.

2. The AWaC Thesis

AWaC treats the workspace as the product object.

A governed Growthub workspace includes growthub.config.json, a Next.js workspace app, dashboards and tabs, widgets and canvas state, governed Data Model objects, custom table views, folder navigation, helper threads, traces and source records, runtime and sandbox environment objects, local and hosted authority hooks, and exportable starter kit assets.

This means the workspace is not a side effect of agent work. It is the controlled environment in which agent work happens.

The result is similar in spirit to infrastructure-as-code, but applied to agent workspaces. The infrastructure is no longer only cloud resources. It is the operating surface where humans and agents coordinate.

3. The Completed UX Primitive: Folders as the User Mental Model

The workspace folders navigation module is the UX proof point for AWaC. The user-facing rail supports folders, dashboard shortcuts, governed Data Model object view shortcuts, folder search and filter, icon and color customization, tree connectors, top-layer add dashboard and add view modals, customize modals, collapsed rail controls, live dashboard routing, and live Data Model object routing.

The key design decision is the divider line. Everything above the divider can become the end-user workspace surface. Everything below the divider can remain administrative, operational, or hidden under the hood.

For end users, the model is straightforward: open the workspace, see folders, open dashboards, open table views, customize permitted navigation, and work inside the exact surfaces the workspace owner exposed.

For super admins, the model is equally important: configure the workspace, decide which dashboards and views are exposed, hide deeper primitives below the divider, keep all state portable in the workspace artifact, and export, fork, sync, and update the workspace over time.

4. Why the Size Matters

The original release proved that a sub-megabyte workspace config can carry real operational state: dashboard canvas state, custom object schemas, navigation folders, widget bindings, helper records, sandbox/runtime rows, governed source records, and policy-relevant configuration.

The current validated workspace config is approximately 688 KB. That scale is the point: a tiny, inspectable artifact can carry a working workspace operating system.

That size is central to the thesis. The artifact is small enough to inspect, diff, fork, export, and reason about, while still being large enough to represent a complete operating workspace.

The May 21 workflow extension preserves that principle. Workflow state is not moved into an opaque external service. It is represented as governed workspace configuration that can travel with the exported starter.

5. Standards Alignment

AWaC is not a replacement for frontier agent standards. It is the workspace substrate that lets those standards become usable in real customer environments.

OpenAI Agents SDK patterns emphasize standardized infrastructure, model-native harnesses, sandbox execution, tracing, and production-ready controls. AWaC provides the local/exportable workspace context where traces, tools, and runtime decisions become visible and governable.

Anthropic MCP standardizes how applications provide context to LLMs. AWaC defines the workspace in which those context connections are exposed, governed, and made useful to humans.

Microsoft Foundry Agent Service emphasizes managed runtime, tools, identity, observability, publishing, security, and lifecycle. AWaC provides a local-first artifact that can map into these enterprise concepts without forcing every user to begin inside an enterprise control plane.

Google Vertex AI Agent Engine and Agent2Agent-style collaboration need shared operating context. AWaC gives multi-agent workflows a shared artifact: dashboards, table views, traces, source records, runtime config, and now workflow orchestration.

6. Why This Is Product-Led

AWaC is not only an infrastructure pattern. It is a product-led growth pattern.

A user can get value locally before signing into hosted authority: download a governed workspace, run it locally, open dashboards, create folders, add permitted object views, customize navigation, use helper support, and operate the workspace as a real app.

Only after local value is proven does hosted authority become necessary: hosted memory sync, knowledge persistence, bridge integrations, enterprise governance, shared authority, managed agents, and team-level trust.

This sequencing matters. It lets users understand and own the artifact before asking them to trust a platform. The workspace itself becomes the activation surface.

7. Governance Model

AWaC separates three layers.

The user surface contains folders, dashboards, table views, helper access, workflow items, and workspace settings. The UI is clean and intentional.

The admin surface lets super admins decide what appears in the user surface, what stays hidden, and how dashboards, views, workflow items, sandbox rows, and governed objects map together.

The agent surface reads the same configuration, traces, memory, object schemas, helper records, and runtime contracts. Agents do not need a separate invented mental model.

This creates alignment between humans, agents, and admins. Everyone operates against the same workspace artifact.

8. Workflow Extension: Builder Items Over Governed Execution

The May 21 release extends the original AWaC model with workflow folder items. A workflow is presented in Builder as a first-class item, while execution remains bound to a governed sandbox-environment row.

This is an extension of the AWaC thesis, not a replacement. The workspace remains the owned artifact. The sandbox row remains the execution primitive. The workflow canvas becomes the specialized layout surface for orchestration config.

The Builder table now lists dashboards and workflows together with clean filtering and compact action menus. Workflows open into a canvas route that resolves the exact governed record and orchestration field.

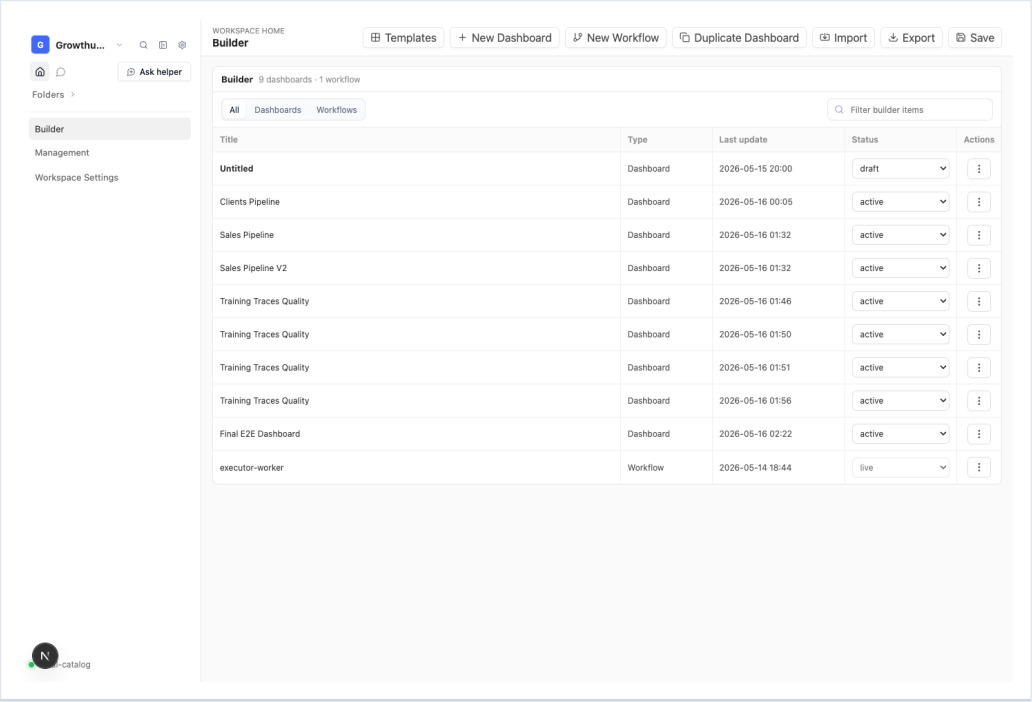

Figure 2. Builder now supports dashboards and workflow items as part of one workspace creation surface.

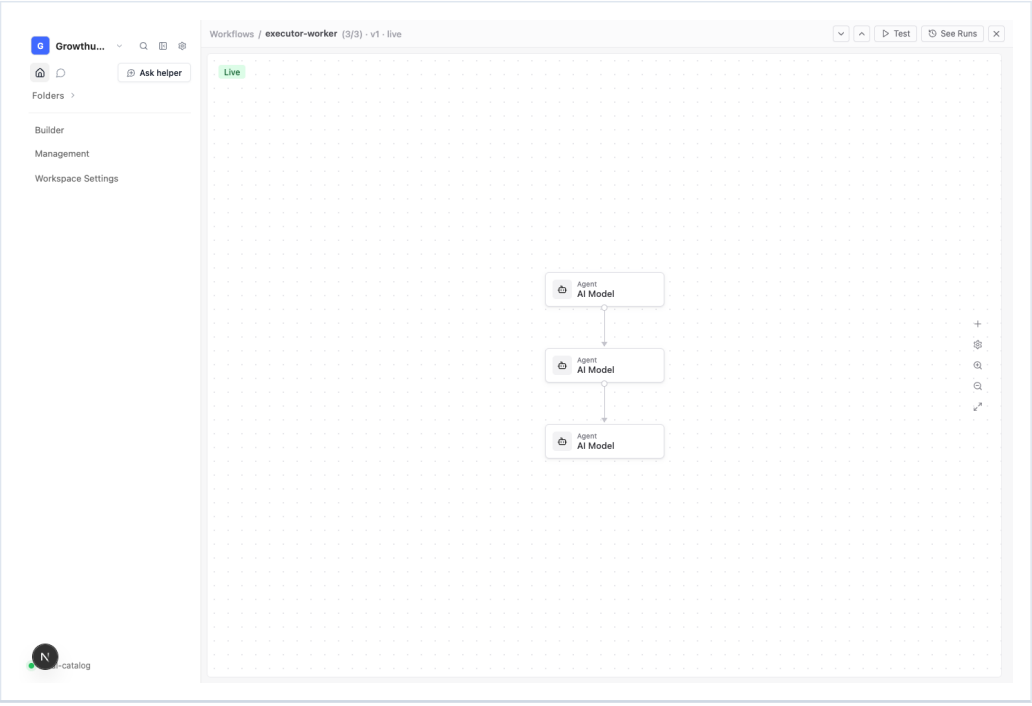

Figure 3. Workflow canvas state is an executable workspace surface over governed orchestration config.

9. Dashboard Widget Configuration Reuse

Workflow node configuration follows the existing dashboard builder grammar. A selected canvas item opens a right-side configuration panel with compact fields, source binding, filters, sorting, placement metadata, and config bindings.

This reuse is important. It means workflow nodes are not a separate visual language. They extend the same no-code configuration model already used by dashboard widgets.

The dashboard sidecar anchors the workflow sidecar decisions: white backgrounds, restrained borders, low-radius controls, compact labels, dropdowns, filter primitives, and source-aware editing.

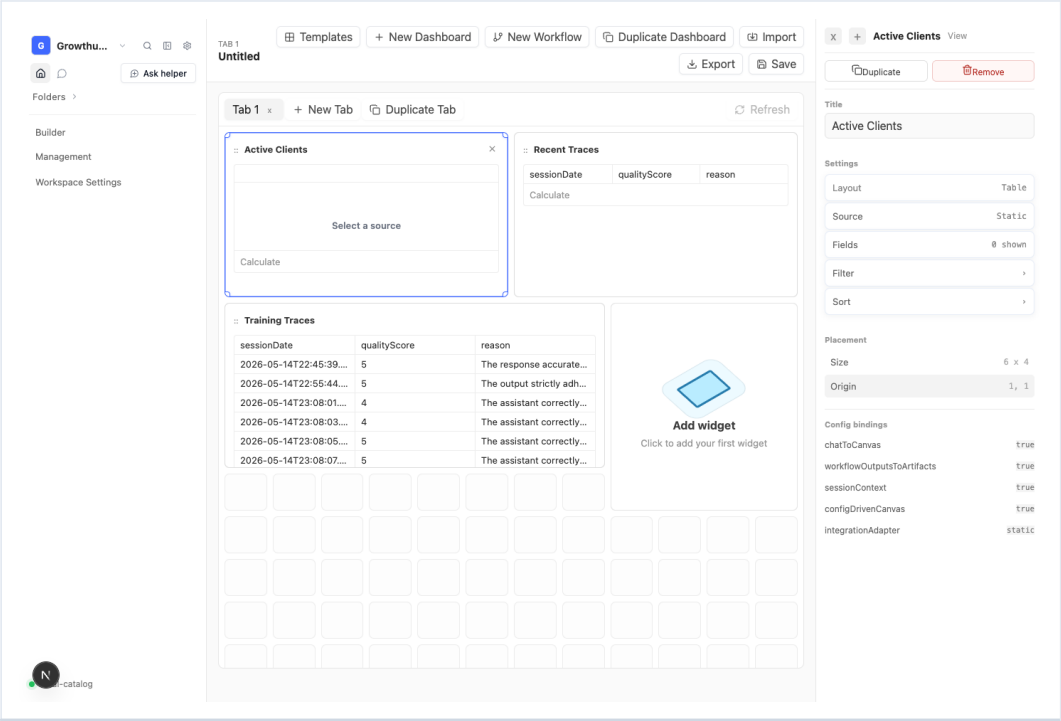

Figure 5. Dashboard widget configuration is the existing no-code sidecar grammar reused by workflow node configuration.

10. Workspace Helper Direct Support

Workspace Helper remains the assistant layer over the governed workspace grammar. It can draft dashboards, create objects, edit views, repair workspace state, and support workflow-adjacent workspace authoring.

The helper remains proposal-first. It does not bypass the workspace mutation contract. Proposed changes must still apply through governed workspace APIs.

In the complete user journey, Helper proposes or explains, Builder surfaces dashboards and workflows, Data Model exposes governed records, and the workflow canvas edits executable orchestration state.

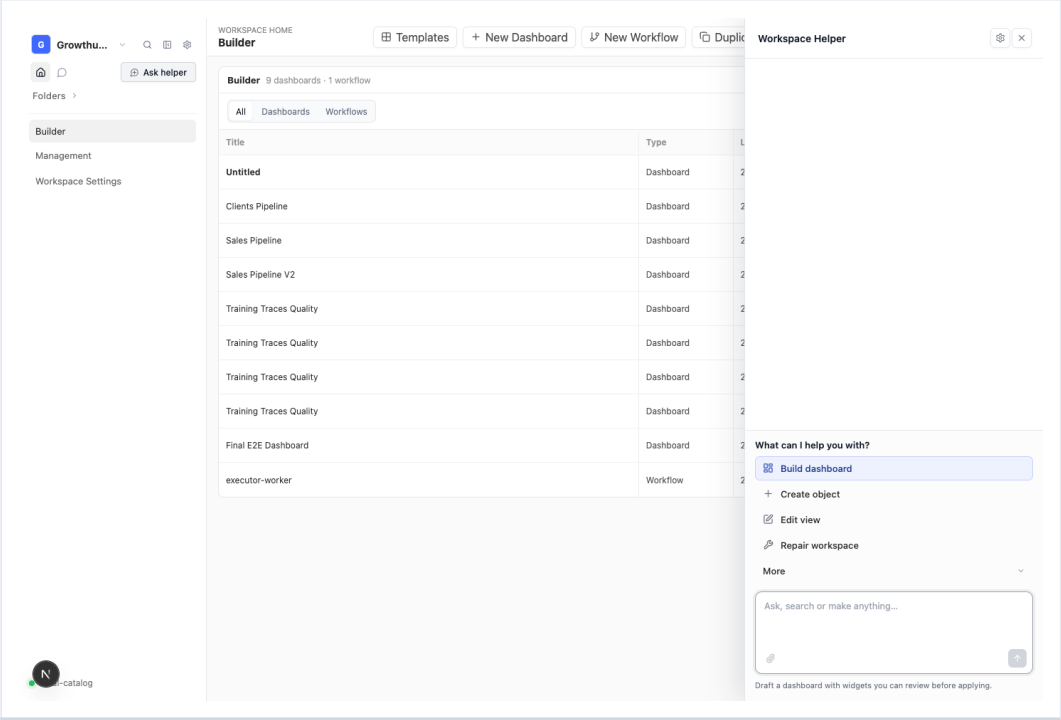

Figure 6. Workspace Helper remains the proposal-first support surface for Builder, Data Model, and workflow-adjacent changes.

11. Local AI Model and Sandbox Execution Support

Local model support is grounded in the sandbox-environment row rather than a workflow-only setting. AI Model nodes are user-facing projections of sandbox-backed execution config.

The sandbox row carries adapter, run locality, runtime, model reference, network policy, prompt, instructions, lifecycle, version, and output fields. The workflow node can remain simple while the runtime stays deterministic.

This preserves the AWaC principle that execution authority belongs to governed runtime primitives, not to decorative graph JSON.

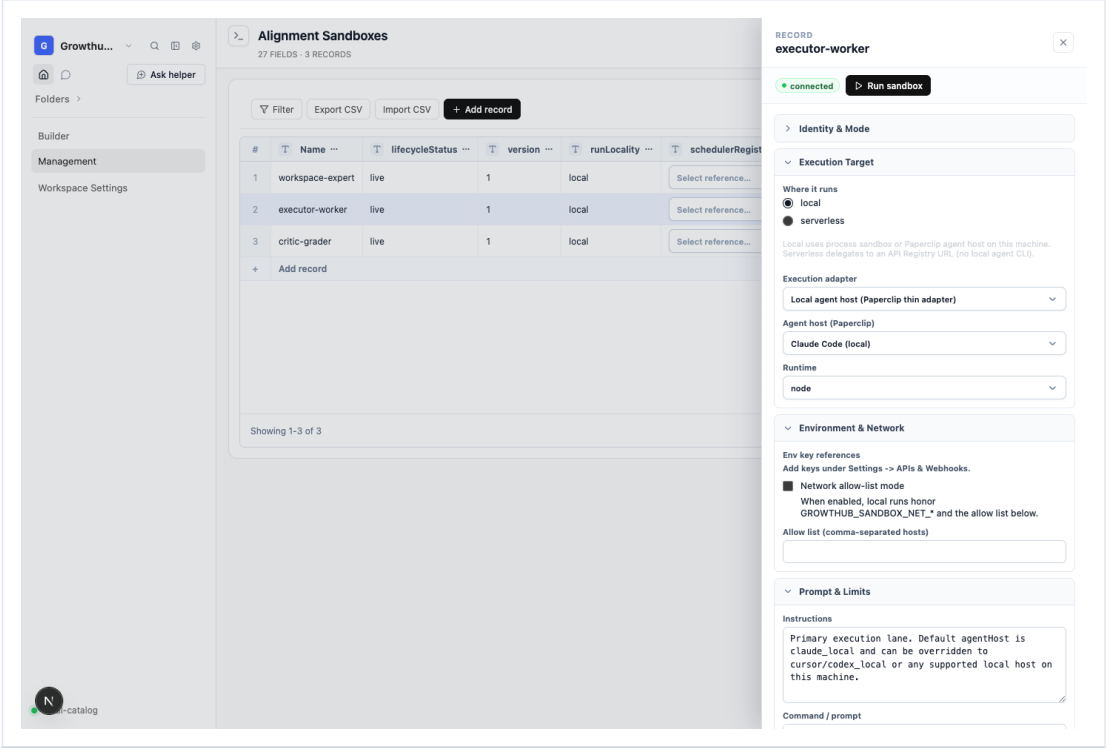

Figure 7. Local AI Model support is grounded in sandbox-environment execution fields and local-agent-host adapter configuration.

12. Draft, Test, Publish Safety

Workflow editing introduces draft/live separation. Save Draft writes editable state without changing live orchestration. Test executes the exact saved draft through the sandbox run path. Publish promotes only a successful tested draft into live orchestration config.

This keeps the workspace safe. Users can configure graph structure, node bindings, data actions, local model behavior, and runtime fields without mutating live execution until a successful test proves the draft.

The publish guard is deterministic, not optional UI preference. Live execution config remains protected by the workspace safety contract.

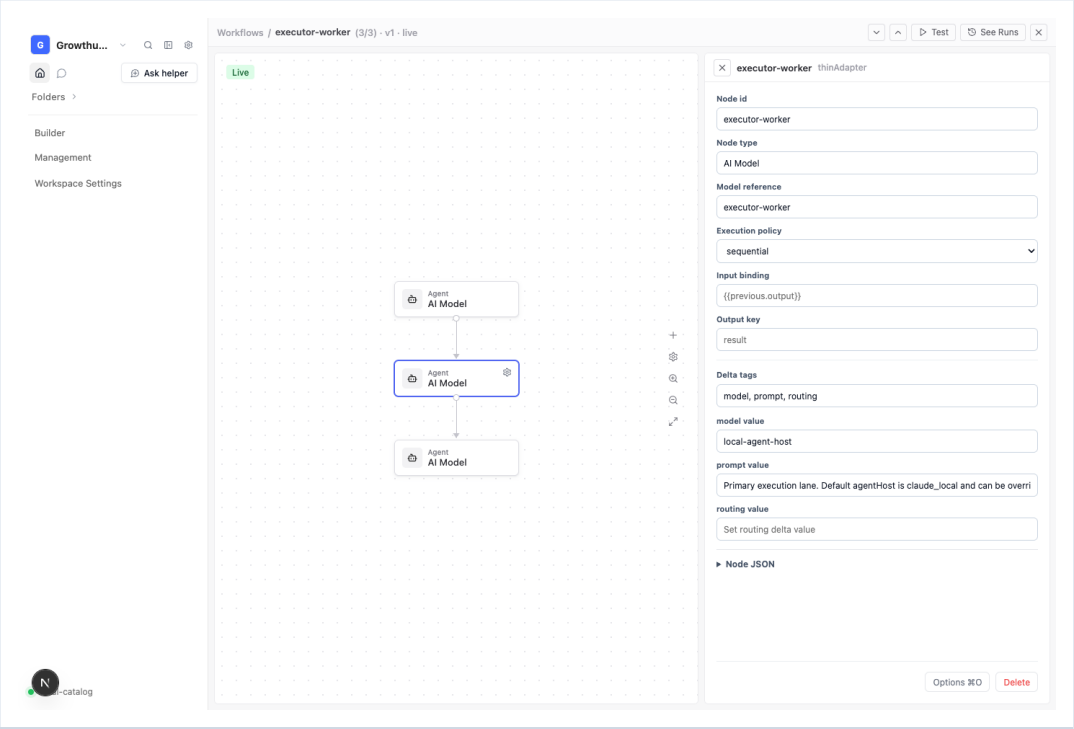

Figure 8. Workflow sidecars expose real configuration and delta-bound values without bypassing draft/test/publish safety.

13. Use Cases

AWaC supports client delivery portals with only client-safe dashboards exposed, internal operations consoles organized by workflow folders, sales and support workspaces, agent service dashboards backed by traces and Data Model objects, governed no-code apps exported as Next.js workspaces, local-first enterprise pilots, vertical SaaS starter kits, agency handoffs, compliance-sensitive workspaces, and training or QA environments where agent traces remain inspectable.

It supports a clean PLG journey: a free user can download a workspace, run it locally, organize operating folders, create dashboard shortcuts, expose only the object views they care about, and immediately feel ownership. As the workspace grows, the same artifact can connect to hosted authority, memory, knowledge, bridge integrations, and release workflows without changing the user mental model.

With workflow support, those use cases expand from passive workspace surfaces into governed executable flows: sandbox-backed AI model runs, HTTP actions, data actions over workspace objects, filters, delays, human input, and publish-gated workflow changes.

14. Technical Validation

The original AWaC release was validated through local runtime testing, byte-for-byte starter copyback, npm release, Release OSS workflow success, and fresh npm smoke using the exported starter.

The workflow extension was validated through the same standard: local app smoke testing on localhost, real /api/workspace persistence checks, canvas and sidecar UI validation, sandbox run validation, npm release through the official Release (OSS) GitHub Action, and fresh exported workspace inspection.

The release confirmed that the distributed starter includes workflow route files, orchestration graph components, sidecar routing, graph runner support, sandbox execution integration, dashboard builder support, helper support, local model configuration surfaces, CLI export paths, and governed fork primitives.

15. Strategic Conclusion

Agentic AI needs more than powerful models. It needs owned environments where humans, agents, tools, memory, knowledge, workflows, and governance can coordinate.

Agent Workspace as Code provides that environment.

The completed Growthub Local workspace starter proves that a full agentic workspace can be local-first, exportable, governed, user-configurable, admin-controllable, traceable, connected to hosted authority when needed, and small enough to inspect and fork.

The workflow extension strengthens the same thesis. It turns executable orchestration into another governed workspace surface, while preserving the original AWaC invariant: the workspace remains the portable, governed, user-owned substrate for production agentic AI work.

Architecture Reference

Core thesis	The workspace is the owned, portable, governed artifact.
User surface	Folders, dashboards, views, Helper, Builder, and workflow items.
Frozen snapshot	The validated workspace config is approximately 688 KB while carrying dashboards, Data Model views, folders, helper state, sandbox execution, and workflow orchestration.
Execution surface	Governed sandbox-environment rows and sandbox-run API.
Workflow live state	orchestrationConfig.
Workflow draft state	orchestrationDraftConfig and draft metadata.
Safety invariant	Publish is blocked until the exact saved draft returns a successful sandbox run.
Distribution invariant	The capability ships in the npm-distributed starter kit, not only in local source.

Closing Sentence

Agent Workspace as Code is the portable, governed, user-owned substrate for production agentic AI work. The workflow extension keeps that sentence true while adding governed executable orchestration to the same workspace artifact.